

BIOFLOW STUDIO

把一组科研材料 带成一条可追溯的分析路径

生命科学智能体工作台 · 前端改进与工程闭环

项目汇报 / 2026.09

面试官先看懂什么？

研究员上传材料后，如何知道
Agent
依据什么、正在做什么、结果从哪里来。

核心主题：可理解的流程实例，不包装成全量生产平台。



真实登录页 · DNA 粒子背景



先用一条主线解释项目，再拆工程证据

每一段都回答一个面试官自然会追问的问题。

01

题目怎么理解

从陌生产品到可运行闭环

02

流程实例怎么跑

材料 → 证据 → 计划 → 结果

03

工程为什么可信

RAG、SSE、鉴权、存储与计算

04

前端亮点在哪里

画布、状态、局部反馈与响应式

05

怎样现场验收

真实能力、模拟边界与下一步



我把作业要求翻译成四个可验收的问题

不是先堆功能，而是先明确每个要求的证据形式。

01

理解陌生 AI 产品

把 Biomni 的用户路径拆成任务、材料、技能、运行与结果

产品地图 + 主线取舍

02

形成前后端闭环

让一条研究流程有输入、接口、状态与产物

流程实例 + API

03

做有价值的前端交互

把等待、审查、执行和结果放进同一任务上下文

SSE + 画布 + 预览

04

AI 辅助但保持质量

用类型、鉴权、测试和边界约束辅助开发输出

验证记录 + 边界

BioFlow Studio 解决的是“研究过程失联”

材料、判断、执行和结果原本分散；我把它收进一个任务上下文。



一句话：把“问一个科研问题”变成一条能回看、能暂停、能解释的流程实例。

先说明什么已接入，再说明什么只是流程实例

这一页是全篇的口径基线：不把模拟过程包装成真实生产能力。

已接入

- 文件解析与预览
- Bearer + RBAC
- Qdrant / BM25 / RRF
- SQLite 作业与 PyDESeq2

流程实例

- 演示任务中的 SSE 事件
- 画布节点状态
- 结果页候选基因
- 账号与示例材料

未验收 / 下一步

- 真实 OCR 模型
- cross-encoder
- 完整 Mol*
- 多租户与生产调度

演示时：页面会显示流程实例；真实链路以接口、日志与产物为准。

用一个 RNA-seq 分析实例贯穿后面的技术解释

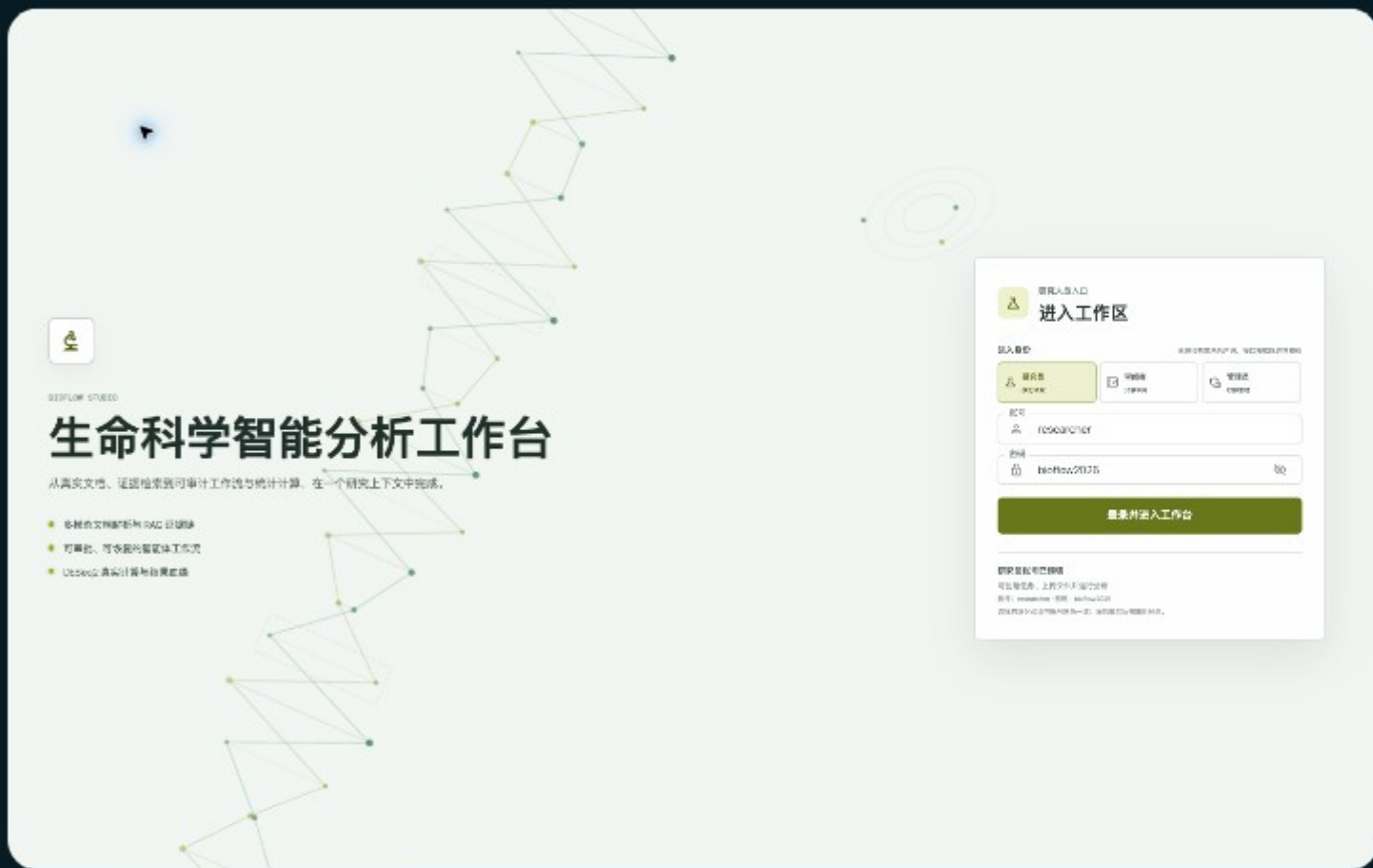
演示材料是合成练习输入；流程协议、鉴权和统计执行机制才是要说明的工程重点。



后续页不按菜单讲，而是逐段解释：为什么这一步需要它、接口怎么连、用户能看见什么。

首屏 DNA 粒子，先建立研究上下文

角色选择与动态背景同时出现，用户先确认身份再进入任务。



DNA PARTICLE STUDY

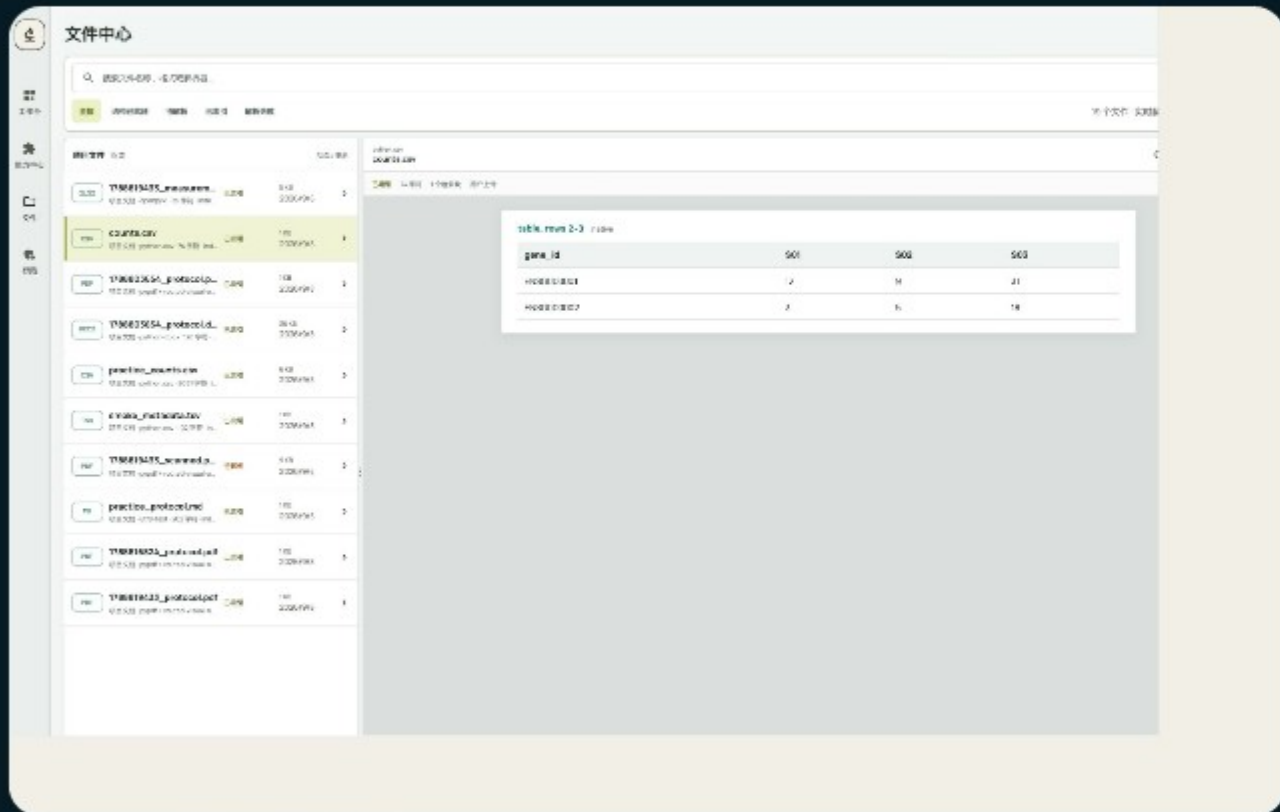
真实页面的动效语言

- 01 斜向双螺旋骨架
- 02 离子化发光节点
- 03 低干扰的慢速漂移

动态只服务状态感知，不抢登录动作。

文件先被理解，才允许参与检索和计算

系统不把所有文件当成纯文本：格式路由决定解析方式、定位信息和后续限制。



CSV / TSV

结构字段与样本对齐

XLSX

工作表与单元格

DOCX

段落与表格

PDF / 图片

文本层或 OCR 路由

展示规则：截图完整展示，不裁切掉文件内容或侧栏。

RAG 的目标不是“找一段话”，而是让回答可回到原文件

这里展示实际接口链路：任务 fileIds 限定检索范围，返回片段、定位和各路分数。

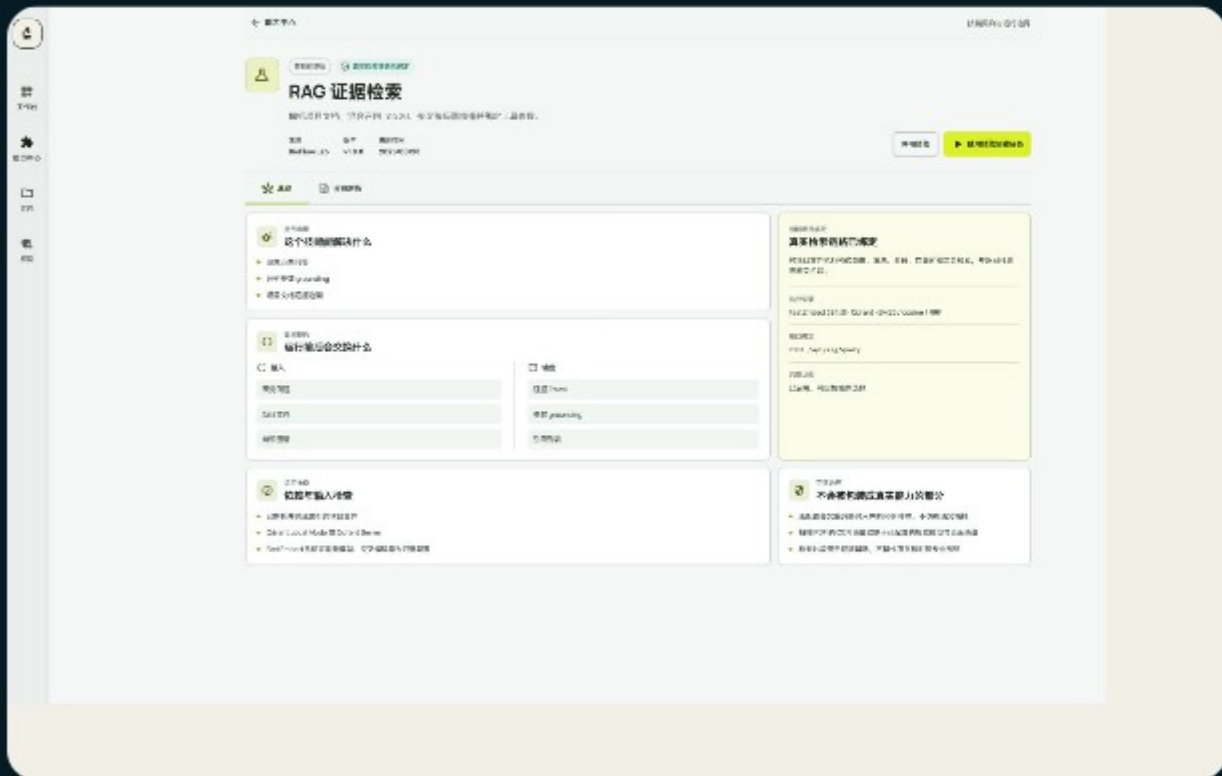


接口事实

POST /api/rag/query → Research Service /search → 返回
fileName、locator、denseScore、bm25Score。
当前没有启用 cross-encoder 时，页面明确显示 RRF，不伪称精排。

用户看到的不只是答案，而是答案使用了哪些材料

真实文件检索与演示任务 Trace 必须区分：前者有服务端来源，后者用于讲交互结构。



一条 Trace 至少包含

- **来源** 文件名 / 页 / 表 / 段落
- **过程** 解析 → 索引 → 召回 → RRF
- **边界** 检索不是统计结论

目的是让研究员核对，不是让模型替代判断。

Agent 在这里负责组织步骤，不负责越过人的研究判断

输入是问题、澄清信息与证据；输出是可审批计划；高成本执行前必须停在审批门。



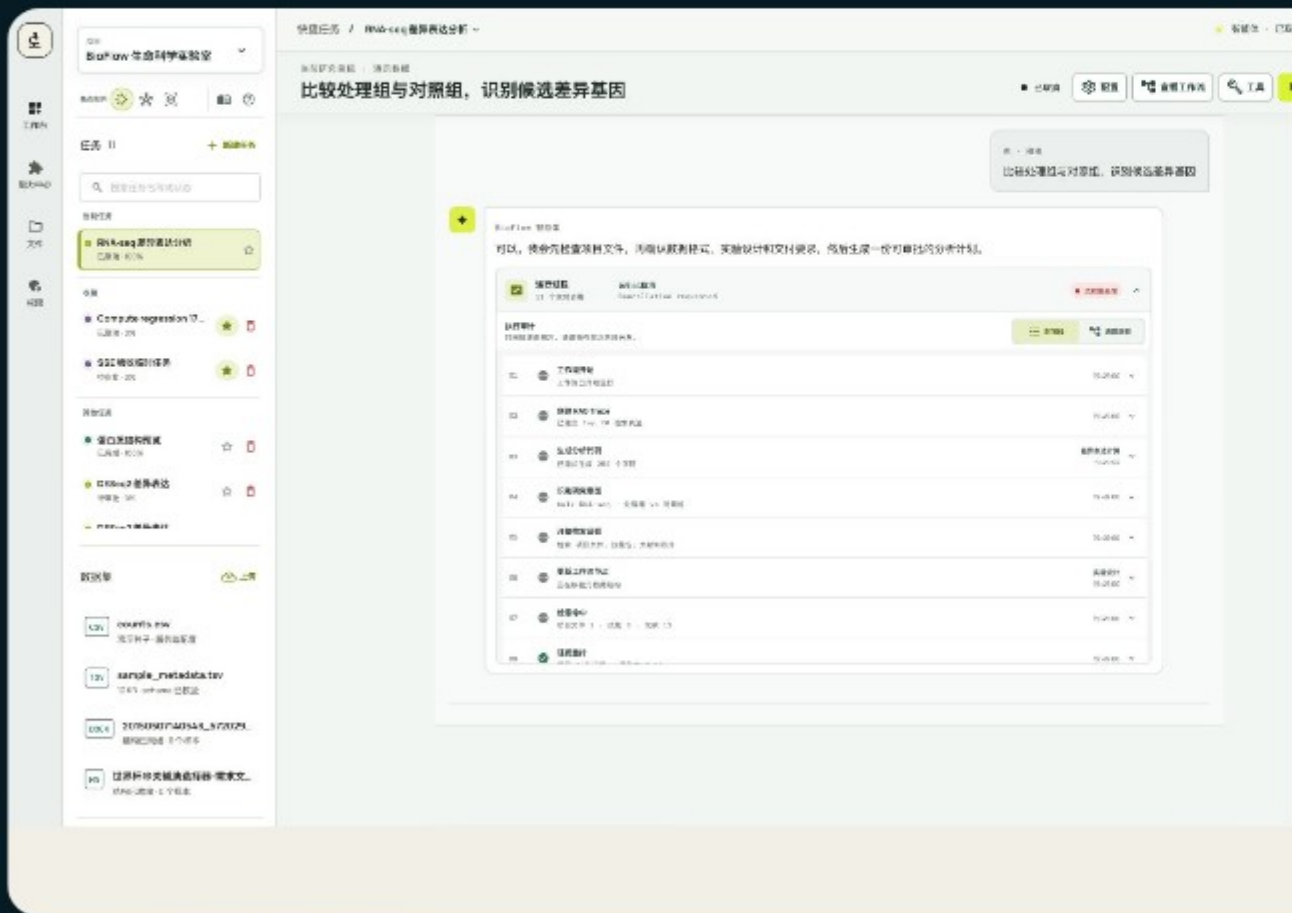
服务端计划接口

**POST /api/agent/plan 以 JSON 或 SSE
返回；失败事件包含失败阶段与可操作 hint。**

演示任务可以回放过程实例；真实供应商调用是否成功由运行时配置和接口状态决定。

用户不该只看到“加载中”，而该知道当前卡在哪一步

计划流与计算流使用相同的事件协议，前端把每一条事件变成可展开的状态。



plan.started

校验输入

worker.started

等待科研
Worker

plan.waiting

每 8 秒心跳

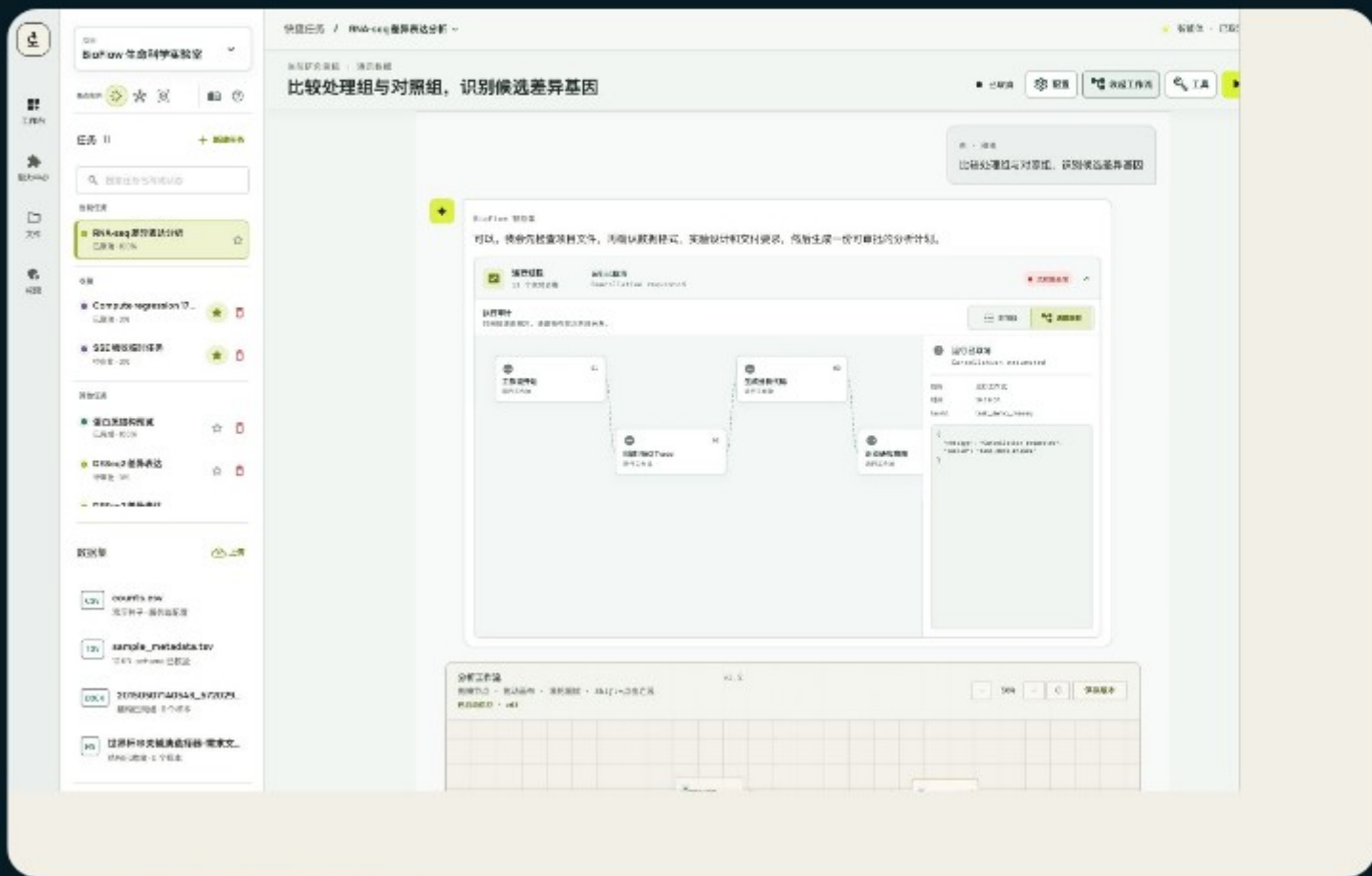
plan.completed

写回任务快照

为什么不用 EventSource?
它不便于携带 Bearer Header。

workflow画布把计划变成可以观察和调整的任务结构

它不是替代后端调度器，而是给用户一个观察依赖、调整布局 and 查看运行状态的表面。

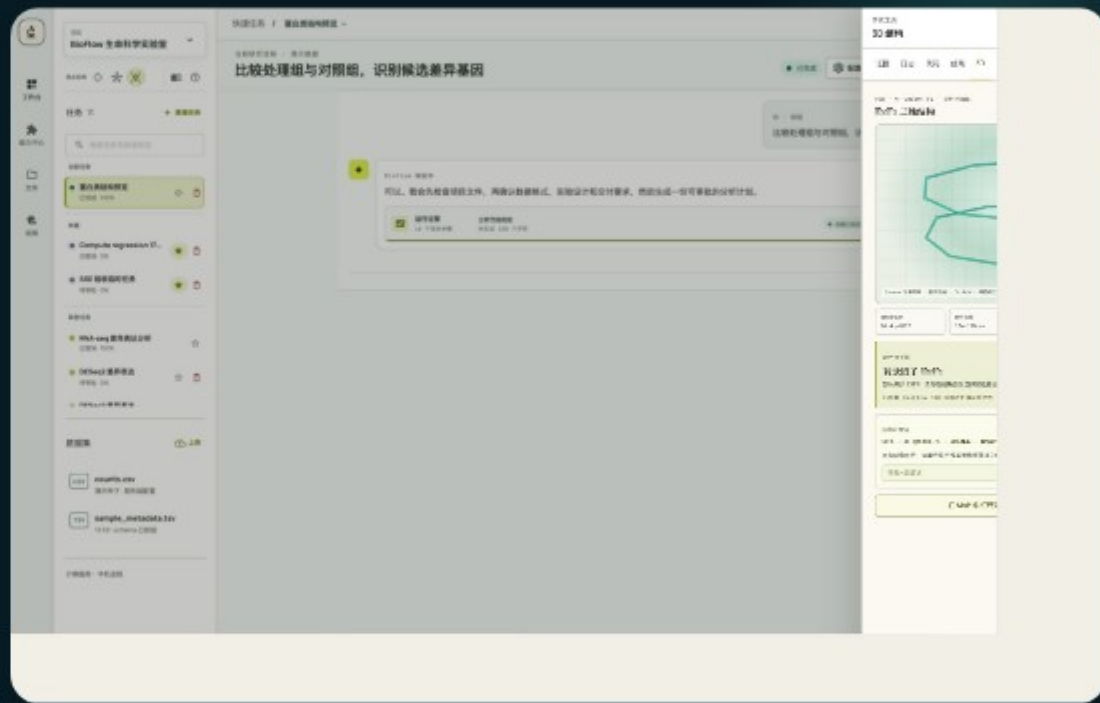
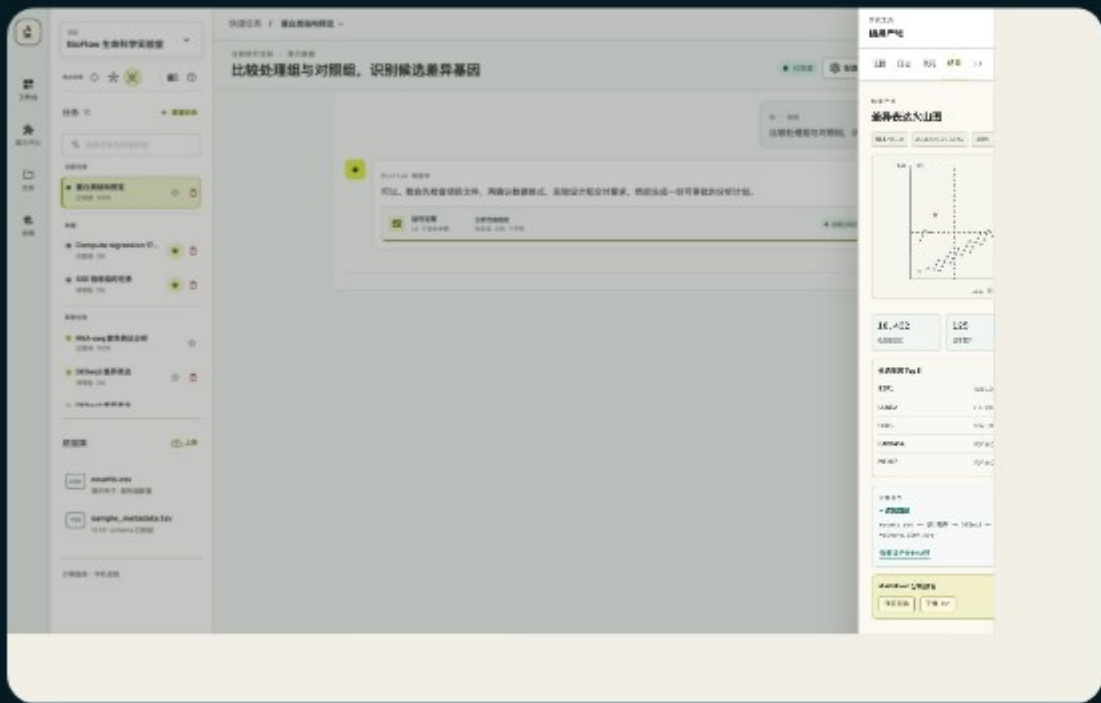


- **拖拽** 节点位置独立保存
- **平移 / 缩放** 查看大流程
- **连线** 表达节点依赖
- **revision** 刷新后恢复布局

前端位置和业务状态分离，避免“拖一下就改了任务结果”。

结果页负责把“数值”连接回“过程”，不是为了堆更多图

火山图、报告、代码和结构预览服务于复核与后续解释。



统计结果 → 候选基因 → 来源节点 / 报告 / 轻量结构预览

浏览器拿到的是短期访问令牌，不是模型或 Worker 密钥

所有浏览器请求先经过 Next.js BFF；BFF 再用服务器专用令牌访问研究服务。



Token: 8 小时有效；服务重启 nonce 改变后旧 Token 失效；写请求校验 Origin。

本地交付不是“没有数据库”，而是把不同类型的数据放到合适的位置

面试优先命令启动；生产迁移目标已经明确写入 DDL 和架构文档。



对象目录

原文件



SQLite WAL

作业 / 文档记录



Qdrant Local

向量 + Metadata



JSON

任务 / 布局 / 目录

生产迁移方向：PostgreSQL + 对象存储 + Qdrant Server + 专用队列。

取舍：面试可运行优先，不把本地文件存储包装成生产多租户。



题目



路径



技术



体验



验收

真实计算和模拟流程实例要分开讲

PyDESeq2 Worker 可以对合成练习输入实际拟合；演示任务的页面状态不等于每次都已完成在线计算。



实际验证记录

8 个合成样本、150 个基因实际执行 PyDESeq2；生成 CSV、PNG、报告与脚本。
限制：结果用于工程链路验收，不输出医学或真实生物学结论。

这里不再说“三栏工作台”，因为当前界面并不是固定三栏

真实合同是：大屏保留任务导航与内容区；检查器按需打开；中小屏改用抽屉或浮动工具。



纠正旧版表述：不是固定三栏，而是按可用空间调整任务导航、检查器与工具入口。

AI 辅助开发的价值不在“生成得快”，而在能否建立约束和验证



本项目的关键不是“AI 写了多少代码”，而是没有把 Token、Worker、错误状态和能力边界留给前端猜。

面试官不必相信 PPT，可以沿着这 6 步自己判断

每一步都有页面入口、接口协议或产物作为证据。

- | | | |
|----|-----------|---------------------|
| 01 | 登录研究员 | 角色一致性与 Bearer Token |
| 02 | 打开文件中心 | 核对解析器、索引状态与原文 |
| 03 | 发起 RAG 查询 | 查看来源片段和 RRF 分数 |
| 04 | 生成计划 | 展开 SSE 当前阶段和等待对象 |
| 05 | 打开工作流 | 拖拽、缩放和布局保存 |
| 06 | 运行练习计算 | 查看作业事件和四类产物 |

启动

npm run dev

127.0.0.1:3000

GitHub

用最容易理解的话， 说明一条专业的研究分析路径

○

我完成的不是一组静态页面，而是：

- **产品** 从参考产品拆出主链和取舍
- **工程** 把接口、鉴权、存储和事件接起来
- **体验** 把等待、审阅、执行和结果放进同一上下文
- **边界** 真实能力、流程实例与后续工作分开说

